

অপারেটিং সিস্টেমের ভূমিকা

(Introduction to Operating Systems)

প্রফেসর চেস্টার রেবেরো

(Prof. Chester Rebeiro)

কম্পিউটার বিজ্ঞান ও প্রকৌশল বিভাগ

(Department of Computer Science and Engineering),

ইন্ডিয়ান ইনস্টিটিউট অফ টেকনোলজি, মাদ্রাস

(Indian Institute of Technology, Madras)

সপ্তাহ – 06

বক্তৃতা – 28

মিউটেক্স

(Mutex)

হ্যালো। এই ভিডিওটিতে আমরা Mutex দেখব, যা একটি critical section সমস্যার সমাধান করতে ব্যবহৃত হয়।

(স্লাইড সময় : 00:25 দেখুন)

সুতরাং আমরা spinlock গুলির সাথে শেষ ভিডিওতে যেখানে আমরা শেষ করেছি সেখান থেকে শুরু করব। মূলত, spinlocks এর প্রধান চরিত্র হল এটা busy waiting এ ব্যবহার করা হয়। আমরা দেখেছি যে একটি লক পাওয়ার জন্য, একটি while লুপ ছিল এবং যখন while লুপ xchg instruction ধারাবাহিকভাবে চালু ছিল এবং যখন while loop এ শুধুমাত্র xchg নির্দেশের return value 0 হলে লুপটি প্রস্থান করে। তাই এই busy waiting আদর্শগতভাবে প্রয়োজন নেই। মূলত, ব্যস্ত অপেক্ষার ফলে CPU cycle নষ্ট হয়ে যায়, যার ফলে performance degradation এর পাশাপাশি মেমরির বিশাল অপচয়ও হতে পারে। সুতরাং, আমরা কোথায় স্পিনলক ব্যবহার করতাম? স্পিনলকগুলি কার্যকর যখন আমরা সংক্ষিপ্ত critical section ব্যবহার করি এবং আমরা জানি যে আমাদের অপেক্ষা করার সময় খুব বেশি সময় অপচয় করতে হবে না। সুতরাং উদাহরণস্বরূপ, যদি আমরা একটি shared counter বৃদ্ধি করতে চাই তাহলে আমরা সম্ভবত একটি spinlock ব্যবহার করতে পারি বা অন্য একটি

array element access করতে হয় তাহলে একটি spinlock পছন্দ করা যায়। মূলত, এই এতে অত্যধিক overhead হবে না। অতএব, এমনকি যদি আরেকটি প্রক্রিয়া counter access করা হয় আমরা নিশ্চিত যে প্রক্রিয়া অনেক সময় ব্যয় করে counter বৃদ্ধি করতে। অতএব, অপেক্ষায় থাকা প্রক্রিয়ায় অনেকগুলি cycle নষ্ট করতে হবে না critical section এ প্রবেশ করার জন্য। যাইহোক, স্পিনলকগুলি কার্যকর হয় না যখন অপেক্ষা অনিবার্য হয় বা এটি খুব দীর্ঘ সময় নেয়। উদাহরণস্বরূপ, যদি কোনও page fault থাকে এবং মেমরির একটি page তৈরি করে যা hard disk থেকে main memory তে লোড হয়। সুতরাং, এটি একটি যথেষ্ট দীর্ঘ সময় নেবে এবং আমরা প্রসেস আসলে এই সমগ্র অপারেশনের সময় CPU cycle নষ্ট করুক চাই না। এই ক্ষেত্রে, আমরা একটি Mutex নামক একটি ভিন্ন construct ব্যবহার করি।

(স্লাইড সময় : 02:43 দেখুন)

এখানে এভাবে দেখানো হয়েছে কিভাবে mutex প্রয়োগ করা হয়। তাই, এটি আবারও xchg নির্দেশের উপর নির্ভর করে যা ব্যবহার করা হয় spinlock এর মতোই, আমাদের কাছে লক এবং আনলক এবং একটি memory location রয়েছে যা সমস্ত প্রক্রিয়াগুলির মধ্যে share করা হয়। এখন, critical section পেতে হলে, একটি লক চালু করতে হবে এবং এই লক ফাংশনে আমাদের একটি লুপ থাকবে। সুতরাং, এই spinlock ক্ষেত্রে যেমন xchg নির্দেশটি while লুপের মধ্যে প্রয়োগ করা হয় এবং এই নির্দেশটি হয় 0 বা 0 নয় এমন কোনও মান বা সাধারণত 1 return করবে। সুতরাং যদি মান 0 এর সমান হয়, তাহলে আমরা লুপ থেকে প্রস্থান করব এবং তারপর প্রক্রিয়াটি লক অর্জন করবে এবং critical section এ চালানো হবে। যাইহোক, যদি xchg একটি মান return দেয় যা 0 নয় তবে আমরা এই else অংশে যেতে পারি এবং এই sleep ফাংশনটি execute করতে পারি। এখন, এই sleep ফাংশন প্রক্রিয়া running state থেকে blocked state নেবে হবে। মূলত, প্রক্রিয়া একটি বিশেষ অপারেশন পৌঁছানোর জন্য অপেক্ষা করছে। সুতরাং, যতক্ষণ না এই অপারেশন আসে প্রক্রিয়াটি কোনো CPU সময় পাবে না। এখন, এই ঘটনাটি যার জন্য sleep অপেক্ষা করছে, সেটি wake up ইভেন্ট। সুতরাং, যখন অন্য প্রক্রিয়াকে wake up করতে হয় তখন এটি sleeping process টিকে blocked state থেকে ready queue তে রাখে। এখন, যদি এটি ভাগ্যবান হয় যখন এটি xchg নির্দেশটি চালায় তখন এটি 0 পেতে পারে এবং এটি critical section প্রবেশ করবে। অন্যদিকে, যদি এটি ভাগ্যবান না হয় তবে এটি xchg instruction চালানোর সময় যদি শূন্য নয় এমন মান পায় তবে এটিকে sleep state এ ফিরে যেতে হবে। এবং এটি অন্য প্রক্রিয়া দ্বারা wake up পর্যন্ত sleeping থাকবে। তাই মূলত, আমরা এখানে দেখি যে spinlock এ busy waiting এর পরিবর্তে, mutexes- এ আমরা পুরো প্রক্রিয়াটি একটি sleeping অবস্থায় রেখেছি। এটি একটি sleeping state এ থাকবে যতক্ষণ না এটি wake up হয় এবং যখন এটি wake up হবে, এটি লকটি আবার চেষ্টা করবে এবং এটি একটি লক অর্জন করলে তা critical section এ প্রবেশ করবে।

(স্লাইড সময় : 05:34 দেখুন)

তাই mutex এর সাথে একটি সমস্যা হল যা Thundering Herd Problem নামে পরিচিত। Thundering Herd Problem দেখা দেয় যখন আমাদের প্রচুর প্রসেস থাকে। সুতরাং, এই প্রসেসগুলির প্রত্যেকটি আমরা অনুমান করি যে একই critical section ব্যবহার করছে এবং critical section প্রবেশ করার জন্য লককে invoke করছে হয় এবং critical section শেষে আনলক চালু করে, যা পরবর্তী সময়ে critical section এর জন্য অপর একটি অপেক্ষারত প্রক্রিয়াকে wake up করবে। সুতরাং, যদি আমাদের sleep mode এ প্রচুর সংখ্যক প্রসেস থাকে, তবে একটি প্রক্রিয়া critical section প্রবেশ করে। সুতরাং, এই প্রক্রিয়াটি যখন আনলক execute করে তখন এটি wake up হয়। এই সমস্ত প্রক্রিয়া যা sleep state থেকে wake up। সুতরাং, এই সমস্ত প্রসেসগুলি তখন blocked state থেকে ready queue তে চলে যাবে এবং নির্ধারিতভাবে এই প্রক্রিয়াগুলি ক্রমানুসারে চালানো হবে। সুতরাং, প্রতিটি প্রক্রিয়া তারপর তার while লুপ চালিয়ে যাচ্ছে এবং xchg ফাংশন চালানো যাচ্ছে। তাই xchg নির্দেশের atomic nature এর কারণে এই সব প্রক্রিয়াগুলির মধ্যে শুধুমাত্র একটি প্রক্রিয়া লক অর্জন করবে এবং অন্যান্য সমস্ত প্রসেসগুলি sleep state এ ফিরে যাবে। সুতরাং, এটা সব সময় চলতে থাকে। তাই প্রতিটি সময়, যখনই একটি আনলক একটি critical section এর শেষে প্রক্রিয়ার দ্বারা আহ্বান করা হয়, এটি wake up invoke করবে এবং এটি সব প্রক্রিয়াগুলি mutex awake করার জন্য অপেক্ষা করবে এবং একটি ছাড়া বাকিরা sleep state এ ফিরে যাবে। একটিমাত্র প্রক্রিয়া লক লাভ করবে এবং critical section এ প্রবেশ কবে। সুতরাং, এর ফলে, আমরা দেখতে পাই যে যখনই একটি wake up চালু হয় তখন সেখানে কিছু context switch ঘটতে পারে, যাতে সমস্ত প্রসেসগুলি execution করে এবং xchg নির্দেশটি আবার পরীক্ষা করে। এখন, পরিবর্তনগুলো hardware এ implement হয়, একটি প্রসেস ছাড়া সমস্ত প্রসেসই critical section এ প্রবেশ করবে। সুতরাং, এখানে সমস্যাটি এবং কেন এটি একটি Thundering Herd Problem বলা হয় কারণ প্রত্যেক সময় একটি wake up আছে, প্রচুর context switch হয় কারণ প্রক্রিয়াকরণের একটি বড় সংখ্যা ready queue তে প্রবেশ করতে পারে এবং starvation হতে পারে।

(স্লাইড সময় 08:19 দেখুন)

Thundering Herd Problem এর একটি সমাধান হল queue এ অন্তর্ভুক্ত mutex গুলি পরিবর্তন করা। mutex এর এই বাস্তবায়নে, যখনই লক চালু করা হয় এবং xchg নির্দেশ প্রত্যাবর্তন করে যা কিছু non-zero হয়, প্রক্রিয়াটি একটি queue এ যুক্ত হয় এবং তারপর sleep হয়। এখন, যখন একটি প্রক্রিয়া আনলককে invoke করে, তখন sleeping প্রক্রিয়াটি queue থেকে সরিয়ে দেওয়া হয় এবং বিশেষভাবে শুধুমাত্র সেই প্রক্রিয়ার জন্য wake up হয়। অতএব, পূর্ববর্তী ক্ষেত্রে যেখানে সমস্ত প্রক্রিয়া wake up হত সেটি হয় না, এই ক্ষেত্রে সেখানে ঠিক একটি প্রক্রিয়া যা wake up হয়। সুতরাং এই প্রক্রিয়াটি P যা এখানে নির্দিষ্ট করা আছে তা sleep থেকে wake up

হবে এবং এটি শুধুমাত্র একটি প্রক্রিয়া যা wake up হয়, এটি এই while লুপে প্রবেশ করে, exchange পরীক্ষা করে এবং সম্ভবত এটি লকটি পায় এবং critical section চালায়। একইভাবে, যখন এটি আনলক করে, তখন এটি পরবর্তী প্রক্রিয়াকে বাছাই করবে যা queue তে অপেক্ষা করছে এবং এটি কেবলমাত্র সেই প্রক্রিয়ার জন্য জেগে উঠবে। এখন, এই দ্বিতীয় প্রসেসটি critical section প্রবেশ করবে।

(স্লাইড সময় 09:42 দেখুন)

সুতরাং, যখন আমরা একটি spinlock এবং mutex মতো primitive এর সমন্বয় সম্পর্কে কথা বলছি, তখন এটি বিবেচনা করা গুরুত্বপূর্ণ যে অপারেটিং সিস্টেমে একটি priority ভিত্তিক অ্যালগরিদম ব্যবহার করা হয়। সুতরাং আসুন আমরা এই বিশেষ পরিস্থিতি বিবেচনা করি। সুতরাং, আমাদের বলা যাক আমরা একটি high priority task এবং একটি low priority task একই data share করে এবং একটি critical section আছে। এখন, আসুন আমরা বলি যে low priority task critical section এ চালানো হচ্ছে এবং এই নির্দিষ্ট সময়ে, critical section এ প্রবেশ করার জন্য অন্য high priority task লকের অনুরোধ করে। সুতরাং, আমরা এখানে যে পরিস্থিতিটি সম্মুখীন হয়েছি তা হল যে low priority task critical section এ execute করা হচ্ছে, এই সময়কালে high priority task একটি লক invoke করে এবং critical section প্রবেশ করতে চায়। এখন, আমরা এখানে যে দ্বিধায় আছি তা হল যে আমাদের একটি high priority task বা অগ্রাধিকার টাস্ক রয়েছে যা low priority task এর জন্য অপেক্ষা করছে। সুতরাং, এটি priority inversion problem হিসাবে পরিচিত হয়। মূলত, আমাদের কাছে গুরুত্বপূর্ণ - একটি কাজ যা গুরুত্বপূর্ণ এবং একটি উচ্চ অগ্রাধিকার দেওয়া হয় এবং এটি একটি low priority task তার execution সম্পূর্ণ করতে অপেক্ষা করছে। এবং যদি আপনি এখানে এই নির্দিষ্ট লিঙ্কটিতে তাকান, আপনি একটি বেশ আকর্ষণীয় ঘটনা দেখতে পাবেন যেখানে একটি priority inversion সমস্যা ঘটেছে একটা রাস্তা খোজার জন্য। তাই, priority inversion সমস্যাটির সম্ভাব্য সমাধানটি priority inheritance হিসাবে পরিচিত। সুতরাং, মূলত এই সমাধানটি যখন একটি low priority task critical section এ কার্যকর করা হয় যে critical section এর জন্য একটি high priority task অনুরোধ করে, কি হয় যে low priority task টি high priority তে বাড়ানো হয় মূলত, low priority task এর priority high priority task এর সমান হয়। Low priority task তারপর এই high priority সঙ্গে চালানো হবে যতক্ষণ না critical section শেষ হয়। সুতরাং, এটি high priority task অপেক্ষাকৃত দ্রুত চালানো হবে তা নিশ্চিত করবে। ধন্যবাদ।